

Course Cheat Sheet

Self-Improving ML Agent with RunPod, MCP, PostgreSQL, and Track A/Track B

Project-specific explanation for the Agent-lightening-RL repository

This document is inspired by the [O'Reilly course](#), but the content is original and mapped to this repository. The examples use this project's RunPod, PostgreSQL, Project MCP, Track A, Track B, Lightning, TRL GRPO, LoRA adapters, and Streamlit workflow.

Contents

1	Introduction to Reinforcement Learning for This Project	3
1.1	Intuition: from one ML run to a learning loop	3
1.2	The RL interaction loop mapped to our system	3
1.3	Policy as a probabilistic workflow decision function	4
1.4	Track A and Track B are different learning problems	4
1.5	Acting versus learning	4
1.6	Key takeaways	5
2	Reward Modeling, Guardrails, and Relative Scoring	6
2.1	How reward functions shape behavior	6
2.2	Reward hacking in this project	6
2.3	The raw data boundary	7
2.4	Absolute scoring versus relative scoring	7
2.5	RULER-style evaluation for this project	7
2.6	Key takeaways	7
3	Agent Trajectories and Track A Rollout Design	8
3.1	What is a rollout?	8
3.2	What is a trajectory?	8
3.3	The Track A agent graph	8
3.4	The PostgreSQL/MCP metadata contract	9
3.5	The execution boundary	9
3.6	From scored trajectories to grouped rollouts	9
3.7	Key takeaways	10
4	Track B Optimization, Checkpointing, and Redeploy	11
4.1	Why stability matters	11
4.2	Group-based optimization	11
4.3	TRL GRPO path used in the RunPod validation	11
4.4	Redeploy and compare	11
4.5	Checkpoint forking	12
4.6	Dashboard as evidence console	12
4.7	Key takeaways	12

5	Teaching the Agent to Master Tools with MCP	13
5.1	Why tool use is hard	13
5.2	Project MCP versus RunPod MCP	13
5.3	Safe tool discovery	13
5.4	Scenario generation for tool learning	13
5.5	RULER for tool evaluation	14
5.6	Production as training data	14
5.7	Key takeaways	14
6	Fresh RunPod E2E Operating Recipe	15
6.1	Required and optional inputs	15
6.2	Clone and run	15
6.3	What the bootstrap does	15
6.4	Expected outputs	16
6.5	Key takeaways	16
	Final Mental Model	16

Section 1: Introduction to Reinforcement Learning for This Project

Core idea: Reinforcement learning gives the project a way to improve agent behavior from complete workflow experience. The optimized object is not a single answer. It is the whole sequence: task lookup, safe metadata use, tool choice, benchmark execution, artifact logging, reward scoring, and later policy improvement.

1.1 Intuition: from one ML run to a learning loop

A normal ML automation script runs once and stops. A self-improving agent turns every run into training evidence.

In this project:

- The agent receives a benchmark task from PostgreSQL/MCP.
- It plans a safe ML workflow without seeing raw rows.
- It executes Track A to produce a tabular model benchmark.
- The workflow is scored into a reward.
- Track B uses the scored trajectory to improve the policy.
- The improved policy can be redeployed and compared against the baseline.

Listing 1: The project feedback loop

```
Task -> Agent actions -> Tool calls -> Artifacts -> Reward -> Policy update -> Better task behavior
```

1.2 The RL interaction loop mapped to our system

State

Task metadata, schema summary, previous tool results, policy profile.

Action

Plan step, MCP tool call, benchmark choice, tracking action.

Environment

PostgreSQL registry, execution boundary, ML libraries, Lightning server.

Reward

Weighted score from task completion, model quality, tracking, sandboxing, guardrails.

Policy

LLM or local policy endpoint deciding how the agent should act.

Rollout

One complete Track A execution for one task and policy version.

Trajectory

Recorded steps, tool calls, artifacts, metadata, and final reward.

1.3 Policy as a probabilistic workflow decision function

The policy is not just choosing words. It is choosing workflow behavior.

Listing 2: Policy preferences before reward feedback

```
policy_options = {
    "inspect_safe_metadata": 0.35,
    "derive_preprocessing": 0.20,
    "run_gbm_benchmark": 0.25,
    "log_to_mlflow": 0.10,
    "finish_without_evidence": 0.10,
}
```

Learning should shift probability toward actions that produce strong evidence and away from short-cuts such as finishing without artifacts.

Listing 3: Policy preferences after reward feedback

```
updated_policy_options = {
    "inspect_safe_metadata": 0.30,
    "derive_preprocessing": 0.22,
    "run_gbm_benchmark": 0.30,
    "log_to_mlflow": 0.15,
    "finish_without_evidence": 0.03,
}
```

The agent is learning a process, not memorizing an answer.

1.4 Track A and Track B are different learning problems

Track	What is trained	What it produces
Track A	A tabular task model, such as LightGBM or sklearn histogram gradient boosting	Benchmark metrics, model artifact, trajectory, reward
Track B	The agent policy model or adapter	A LoRA policy adapter or policy checkpoint

Track A asks whether the agent can complete the ML task safely and produce a measurable result. Track B asks whether the agent policy can learn from previous Track A behavior and make better workflow decisions next time.

1.5 Acting versus learning

Listing 4: Two-phase view

```
Acting phase:
    postgres task -> agent graph -> tool calls -> artifacts -> trajectory

Learning phase:
    scored trajectories -> grouped rollouts -> Track B training -> checkpoint
```

This separation matters because the agent should not update itself while it is still performing a task. It first records evidence, then the optimizer trains from evidence.

1.6 Key takeaways

- RL in this project means improving workflow behavior through recorded task experience.
- The policy is a decision rule over agent actions and tool use.
- Track A produces supervised evidence about ML-task completion.
- Track B uses that evidence to improve the agent policy.
- The unit of learning is the full trajectory, not only the final metric.

Section 2: Reward Modeling, Guardrails, and Relative Scoring

Core idea: Rewards shape what the agent becomes. This project rewards safe, reproducible, artifact-backed ML workflows and penalizes unsafe shortcuts.

2.1 How reward functions shape behavior

The scorer combines several components.

Reward component	What it checks
R_data	Safe metadata/profile artifacts exist
R_model	A real benchmark model trained and produced useful metrics
R_tracking	Experiment metadata and metrics were logged
R_sandbox	Code execution used the intended isolation boundary or declared fallback
R_reproducibility	Plans, metrics, model artifacts, and tracking outputs can be replayed
R_violation	Raw-row leakage and guardrail violations were avoided
R_postgres_tool	PostgreSQL/MCP tools were used correctly

Listing 5: Project-shaped reward sketch

```
reward = (
    0.18 * R_data
    + 0.25 * R_model
    + 0.17 * R_tracking
    + 0.15 * R_sandbox
    + 0.15 * R_reproducibility
    + 0.09 * R_violation
    + 0.01 * R_postgres_tool
)
reward *= task_completion_gate
reward *= max(0.5, tool_validity_rate)
```

The reward says: complete the task, use valid tools, preserve safety, and leave evidence.

2.2 Reward hacking in this project

Reward hacking would happen if the agent learned to look successful without doing the work.

- Claiming a model was trained without writing `benchmark_metrics.json`.
- Reading raw database rows into the LLM prompt to make planning easier.
- Skipping MLflow or tracking artifacts.
- Producing a final answer without running the execution boundary.
- Optimizing only for one metric while breaking guardrails.

The scorer and reviewer guard against those behaviors by requiring concrete artifacts and checking leakage markers.

2.3 The raw data boundary

Listing 6: Safe data access boundary

```
MCP/agent layer:
  sees task metadata, schema summaries, column profiles, target profile

Execution boundary:
  can materialize rows through EXECUTION_DATABASE_URL

LLM policy:
  should never receive raw dataset rows
```

This allows the agent to reason about the task without turning the database into prompt data.

2.4 Absolute scoring versus relative scoring

Absolute scoring asks whether one trajectory received a particular score. Relative scoring asks which trajectory for the same task behaved better and why.

Relative scoring is more useful for open-ended agent workflows because one task can be completed in multiple ways. The same final metric may hide different process quality.

2.5 RULER-style evaluation for this project

For richer future runs, RULER can judge groups of trajectories by comparing task accomplishment, tool choice quality, evidence completeness, error recovery, efficiency, and raw data safety.

Listing 7: Relative trajectory comparison

```
same task
  rollout A -> reward 0.70 -> missed tracking
  rollout B -> reward 0.84 -> complete artifacts
  rollout C -> reward 0.60 -> weak model evidence

Track B learns: prefer B-like behavior.
```

2.6 Key takeaways

- Rewards are the curriculum.
- Safety is part of reward, not an afterthought.
- Raw rows belong only inside the execution boundary.
- Relative comparison is better than isolated grading for agentic workflows.
- A good reward teaches the agent to leave useful evidence behind.

Section 3: Agent Trajectories and Track A Rollout Design

Core idea: A rollout is one complete attempt. A trajectory is the durable record of that attempt. Track A turns ML workflow execution into trajectory data that Track B can learn from.

3.1 What is a rollout?

A rollout is one execution of the current policy on one benchmark task.

Listing 8: Track A rollout path

```
Load task -> profile metadata -> plan benchmark -> execute model training -> track metrics -> review -> score
```

If the same task is run five times with slightly different policy behavior, that produces five rollouts.

3.2 What is a trajectory?

A trajectory is the data structure that records the rollout. It includes task id, policy version, ordered agent steps, tool calls, artifact references, final status, reward, reward metadata, and compact additional histories for multi-agent replay.

Listing 9: Minimal project-shaped trajectory

```
{
  "task_id": "mltask_openml_31_german_credit_baseline",
  "policy_version": "baseline_initial",
  "steps": [
    {"agent_name": "DataEngineerAgent", "action": "prepare_metadata_only_profile_artifacts"},
    {"agent_name": "SandboxExecutionAgent", "action": "execute_real_gbm_benchmark_from_postgres_contract"}
  ],
  "final_status": "completed",
  "reward": 0.843
}
```

3.3 The Track A agent graph

Listing 10: Track A graph

```
DataEngineerAgent
-> GradientBoostingSpecialistAgent
-> SandboxExecutionAgent
-> ExperimentTrackingAgent
-> ReviewerCriticAgent
-> reward scorer
```

Each agent owns one part of the workflow. The final trajectory combines them into one learning example.

3.4 The PostgreSQL/MCP metadata contract

The agent starts with safe metadata, not raw rows.

Listing 11: Safe metadata tools

```
task = postgres_get_task_metadata(task_id)
schema = postgres_get_dataset_schema(task_id)
summary = postgres_get_dataset_summary(task_id)
target = postgres_get_target_profile(task_id)
```

The execution layer separately receives an execution-only data source contract.

Listing 12: Execution-only data contract

```
source = postgres_get_execution_dataset_source(task_id)
# source -> ml_data.openml_31_german_credit
```

That source contract is for code execution, not for prompt context.

3.5 The execution boundary

The ideal boundary is Docker. On some RunPod templates Docker-in-Docker is not permitted, so the project supports an explicit validation fallback.

Listing 13: Declared RunPod fallback

```
export CODE_INTERPRETER_MODE=host_subprocess
```

That fallback is transparent in telemetry. The reward scorer gives full sandbox credit to Docker and partial credit to the declared host subprocess mode.

3.6 From scored trajectories to grouped rollouts

Listing 14: Scoring and grouping Track A outputs

```
python -m src.rewards.scorer \
  --input-dir trajectories/tracka_initial \
  --output-dir trajectories/tracka_initial_scored

python -m src.training.prepare_grpo_dataset \
  --input-dir trajectories/tracka_initial_scored \
  --output data/grpo/grouped_rollouts.jsonl
```

Listing 15: Conceptual grouping

```
trajectory_1 reward 0.84
trajectory_2 reward 0.80
trajectory_3 reward 0.65
  |
  v
grouped_rollouts.jsonl
```

The group becomes training data for policy improvement.

3.7 Key takeaways

- A rollout is one complete execution.
- A trajectory is the replayable record of that execution.
- Track A creates trajectories by running a real ML workflow.
- PostgreSQL/MCP gives metadata to the agent and execution contracts to the runner.
- Grouped rollouts are the bridge from Track A to Track B.

Section 4: Track B Optimization, Checkpointing, and Redeploy

Core idea: Track B turns scored behavior into a policy update. The output is a checkpoint or adapter that can be redeployed and benchmarked against the old policy.

4.1 Why stability matters

Agent policies are easy to destabilize. If updates are too aggressive, the model may forget useful workflow habits, overfit to one task, stop using tools correctly, optimize for reward loopholes, or produce less reproducible artifacts.

Stable improvement means small, evidence-backed policy changes.

4.2 Group-based optimization

Listing 16: Group-relative view

```
Task: German Credit baseline
  rollout A: 0.843
  rollout B: 0.801
  rollout C: 0.612

baseline = group average
advantage = reward - baseline
```

This avoids relying only on an external value critic. The baseline comes from real behavior.

4.3 TRL GRPO path used in the RunPod validation

Listing 17: Track B TRL GRPO command

```
TRAINER=trl_grpo \
REWARD_MODE=hybrid \
NUM_GENERATIONS=4 \
MODEL_NAME=Qwen/Qwen2.5-3B-Instruct \
GRPO_DATASET_PATH=data/grpo/grouped_rollouts.jsonl \
POLICY_OUTPUT_DIR=checkpoints/trackb_trl_grpo_runpod \
bash scripts/gpu/run_02_train_policy_qlora_grpo.sh
```

Output:

Listing 18: Adapter checkpoint output

```
checkpoints/trackb_trl_grpo_runpod/
  adapter_config.json
  adapter_model.safetensors
  adapter_metadata.json
```

4.4 Redeploy and compare

After training, the project can run the same Track A task through two policy endpoints.

Listing 19: Before and after endpoint comparison

```
baseline endpoint -> Track A -> reward report
tuned endpoint   -> Track A -> reward report
```

Listing 20: Comparison report command

```
python -m src.evaluation.compare_policies \
  --policy-dir initial_no_llm=trajectories/tracka_initial_scored \
  --policy-dir baseline_llm=trajectories/tracka_baseline_live_scored \
  --policy-dir trackb_redeploy_llm=trajectories/tracka_tuned_live_scored \
  --output reports/tracka_initial_vs_baseline_vs_trackb_redeploy.md
```

The most recent RunPod validation covered four OpenML tasks. Initial Track A averaged 0.85 across 16 trajectories; baseline live LLM and Track B redeploy live LLM each averaged 0.85 across four tasks with live endpoint rate 1.0. This means no regression and no measured lift on this small gate.

4.5 Checkpoint forking

Listing 21: Checkpoint branch mental model

```
baseline policy
-> trackb_trl_grpo_runpod
  -> redeploy comparison
  -> future RULER/ART/veRL scale-up branch
```

Checkpoint forking lets the project explore new reward modes or datasets without losing a known-good baseline.

4.6 Dashboard as evidence console

Streamlit is not the learning system. It is the evidence console. It should show service health, Track A artifacts, grouped rollouts, Track B checkpoints, before/after benchmark reports, and run/service logs.

Listing 22: RunPod dashboard pattern

```
START_DASHBOARD=1 DASHBOARD_PORT=8503 bash scripts/runpod_bootstrap_e2e.sh
```

Then open the RunPod proxy URL for that port. Use 8503 when 8888 is already occupied by Jupyter.

4.7 Key takeaways

- Track B improves the agent policy, not the tabular ML model.
- Grouped rollouts make policy learning comparative.
- LoRA adapters are the practical checkpoint format for this project.
- Redeploy comparison is the gate that proves the policy did not regress.
- Streamlit should surface evidence, not hide it in terminal logs.

Section 5: Teaching the Agent to Master Tools with MCP

Core idea: Tool use should become learned behavior. MCP gives the agent a standardized tool interface, and trajectories teach the policy when and how to use those tools.

5.1 Why tool use is hard

The agent must decide whether a tool is needed, which safe tool to call, what arguments to pass, whether the result is enough, how to recover when a tool fails, and when to stop and produce the final artifact.

Simple success flags do not capture this. Tool quality is behavioral.

5.2 Project MCP versus RunPod MCP

MCP type	Role
RunPod MCP	Infrastructure management, pod lifecycle, deployment
Project MCP	ML-agent tool layer: PostgreSQL metadata, schema summaries, reward history, execution dataset contracts

The learning loop uses Project MCP, not RunPod MCP.

5.3 Safe tool discovery

The Project MCP layer exposes tools such as:

Listing 23: Safe Project MCP tools

```
postgres_get_task_metadata
postgres_get_dataset_schema
postgres_get_dataset_summary
postgres_get_column_profile
postgres_get_target_profile
postgres_get_execution_dataset_source
postgres_get_reward_history
```

The agent can learn that `postgres_get_execution_dataset_source` is not for prompt reasoning. It belongs to the execution boundary.

5.4 Scenario generation for tool learning

Future tool-learning scenarios can be generated from the available tool catalog.

Listing 24: Project-specific tool-learning scenarios

```
[
  {
    "task": "Plan a safe benchmark using only metadata tools.",
    "required_tools": ["postgres_get_task_metadata", "postgres_get_dataset_summary"]
  },
  {
    "task": "Recover when a column profile request asks for a missing column.",
    "required_tools": ["postgres_get_column_profile"]
  },
  {
```

```
"task": "Prepare execution without leaking raw rows.",  
"required_tools": ["postgres_get_execution_dataset_source"]  
}  
]
```

This turns tool behavior into a curriculum.

5.5 RULER for tool evaluation

For tool-heavy tasks, the judge should compare whether the agent chose the right tool, called it at the right time, passed safe arguments, avoided raw rows, recovered from errors, and produced the expected artifact.

The best trajectory is not always the shortest. It is the one that reaches the goal safely with enough evidence.

5.6 Production as training data

Listing 25: Production flywheel

```
production request  
-> tool calls  
-> trajectory log  
-> offline reward or RULER ranking  
-> Track B training batch  
-> updated policy
```

This is the self-improving flywheel.

5.7 Key takeaways

- Tool use is a learning problem, not just a prompt engineering problem.
- Project MCP standardizes the safe tool interface.
- Tool-call telemetry becomes trajectory data.
- RULER-style ranking can evaluate process quality.
- Production usage can feed the next policy update.

Section 6: Fresh RunPod E2E Operating Recipe

Core idea: A new empty RunPod should need only the GitHub URL, DATABASE_URL, and usually a Hugging Face token for model downloads.

6.1 Required and optional inputs

Required:

Listing 26: Required database secret

```
export DATABASE_URL='postgres://USER:PASSWORD@HOST:PORT/DB?sslmode=require'
```

Optional but recommended:

Listing 27: Optional Hugging Face token

```
export HF_TOKEN='hf_...'  
# or  
export HUGGINGFACE_HUB_TOKEN='hf_...'
```

The Hugging Face token matters because gated/private models may require it, anonymous downloads may rate-limit, and repeated pod rebuilds download model shards again.

6.2 Clone and run

Listing 28: Fresh clone path

```
cd /workspace  
git clone https://github.com/keshavnath1/Agent-lightening-RL.git  
cd Agent-lightening-RL  
git checkout E2E_grpo_4openml_streamlit_evidence  
  
bash scripts/runpod_bootstrap_e2e.sh
```

With dashboard and live endpoint comparison:

Listing 29: Full E2E with dashboard and redeploy comparison

```
RUN_LIVE_POLICY=1 START_DASHBOARD=1 DASHBOARD_PORT=8503 \  
bash scripts/runpod_bootstrap_e2e.sh
```

6.3 What the bootstrap does

Listing 30: Bootstrap stages

```
create .venv  
install dependencies  
ingest four OpenML datasets into the PostgreSQL contract  
start Lightning server  
run Track A  
score trajectories  
prepare grouped rollouts  
run Track B TRL GRPO  
optionally redeploy endpoints  
write reports and logs  
optionally start Streamlit
```

6.4 Expected outputs

Listing 31: Runtime outputs

```
reports/run_logs/  
reports/service_logs/  
reports/tracka_initial_benchmark.md  
reports/tracka_initial_vs_baseline_vs_trackb_redeploy.md  
reports/e2e_grpo_run_summary.json  
reports/trackb_trl_grpo_adapter_metadata.json  
data/grpo/grouped_rollouts.jsonl  
checkpoints/trackb_trl_grpo_runpod/
```

Sanitized reports, grouped rollout data, and trajectory evidence are committed for review. Runtime secrets, local `.env` files, live scratch files, and heavyweight checkpoint weights stay out of git.

6.5 Key takeaways

- Source lives in GitHub branch `E2E_grpo_4openml_streamlit_evidence`.
- Secrets stay in the pod environment or untracked `.env`.
- `DATABASE_URL` is required.
- `HF_TOKEN` is optional but practically important.
- The bootstrap script is the reproducible path from empty pod to E2E evidence.

Final Mental Model

Listing 32: End-to-end learning loop

```
PostgreSQL contract  
-> Project MCP safe metadata tools  
-> Track A agent rollout  
-> trajectory and reward  
-> grouped rollout dataset  
-> Track B policy optimization  
-> checkpoint redeploy  
-> benchmark comparison  
-> dashboard evidence  
-> next improvement cycle
```

The project is not only an ML benchmark runner. It is a small self-improvement loop where safe tool use, reproducible artifacts, and reward-bearing trajectories become the training material for a better agent policy.